

TEL AVIV אוניברסיטת
UNIVERSITY תל אביב

Data sources and data processing

Visualization of Information

Salomon Eisler

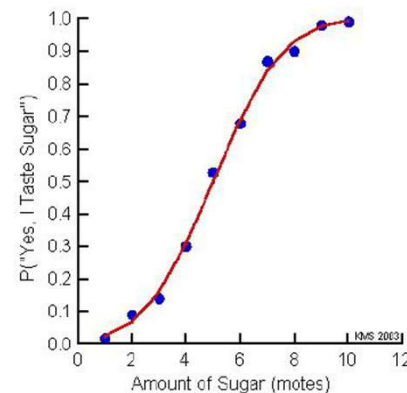
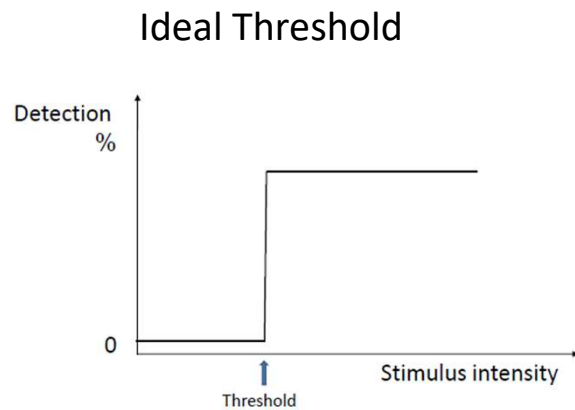
Dept. of Industrial Engineering

Agenda

1. Summary of previous and more details on Visual Entropy
2. Data sources and data processing - Why?
3. Types of Data
4. Data Preprocessing
5. Data Sets
6. Assignment

What is Psychophysics?

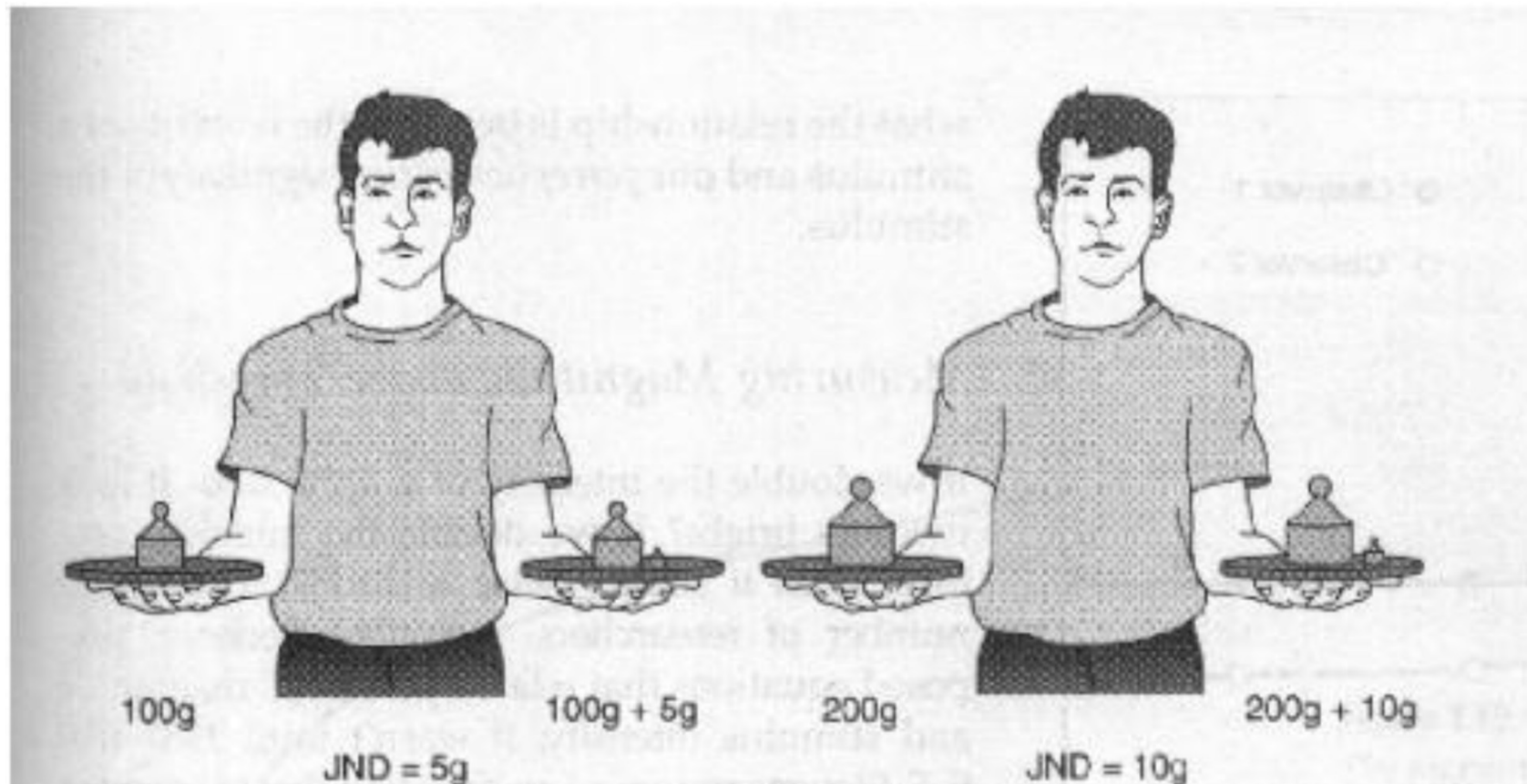
Since its inception, psychophysics has assumed that the human perceptual system is a measuring instrument yielding results (experiences, judgments, responses) that may be systematically analyzed.



Thresholds are defined as percentages. Typically, the threshold will be defined as the value for which the detection rate is 0.5. However, other thresholds may be used, depending on the purpose.

Absolute threshold: the smallest stimulus intensity that can be detected by an organism.

Just Noticeable Difference (JND)



Weber Law

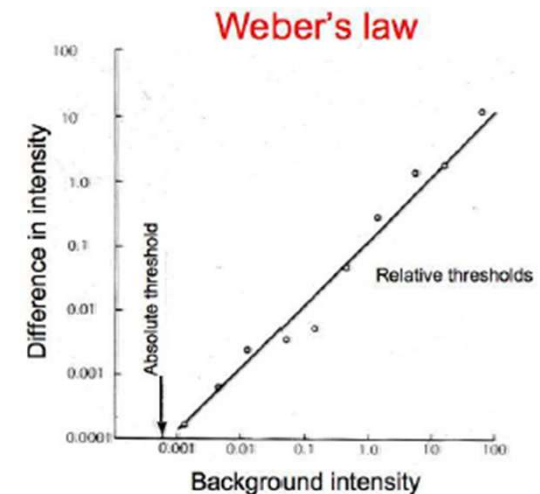
- Just Noticeable Difference (jnd): The minimal change in stimulus intensity required to exceed the differential threshold.
- The jnd is a constant fraction of the stimulus intensity

$$\frac{\Delta R}{R} = k$$

R = Reiz , the German word for stimulus, the standard stimulus.

ΔR = The minimum change in R that could be detected.

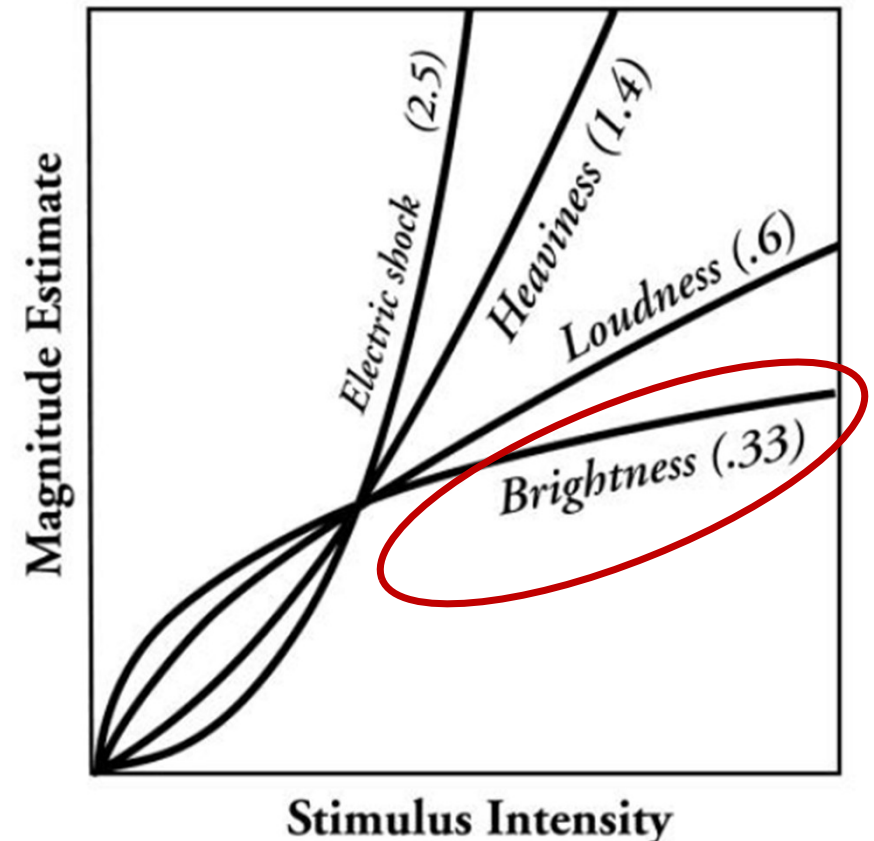
k = constant (as seen earlier, k = 1/20 for lifted weights)



Stevens Law

- Stevens, Stanley Smith (1906-1975)
Harvard psychologist
- Stevens' magnitude estimation method yields power relations for many stimuli
- Ψ is the sensation, ϕ is the stimulus, and α is a sense-specific constant

$$\Psi = \phi^{\alpha}$$



Examples of Stevens' Law in Visualizations

1. Bar Charts (Length)

- **Exponent $n \approx 1.0$:** People are generally **good at judging length**. If a bar is twice as long as another, it's typically perceived as twice as much.
-  Good perceptual accuracy.
-  Recommended for representing quantitative data.

2. Bubble Charts / Circle Size (Area)

- **Exponent $n \approx 0.7$:** People **underestimate differences in area**. If one circle is 4x the area of another, it might only be perceived as 2.5x larger.
-  Can lead to misinterpretation of magnitude.
- Use with caution or add numerical labels.

Examples of Stevens' Law in Visualizations

3. 3D Volume-Based Visuals

- **Exponent $n \approx 0.6$** : People are **even worse at judging volume**, so using spheres or cubes to represent data exaggerates or flattens differences.
-  Generally discouraged.

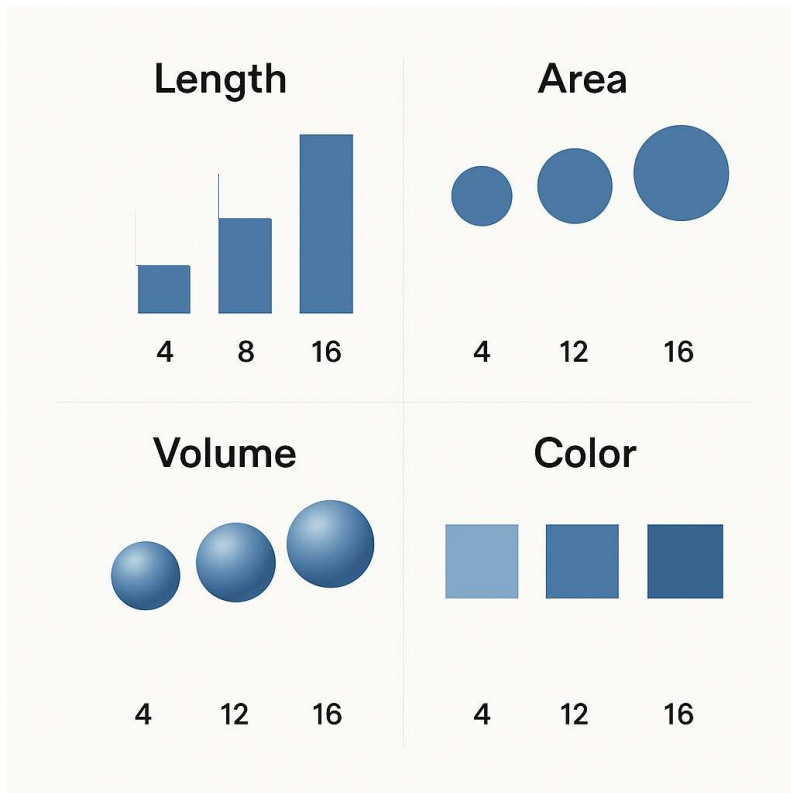
4. Color Intensity / Saturation

- **Exponent $n \approx 0.3-0.5$** : Perception of color brightness or saturation does not scale linearly with the actual value.
- Use **color gradients** carefully, preferably for categorical or ordinal data, not precise values.

5. Position on a Common Scale (e.g., Dot Plots)

- **Exponent $n \approx 1.0$** : Like bar length, **position** is perceived accurately.
-  Excellent choice for comparing values.

Examples of Stevens' Law in Visualizations

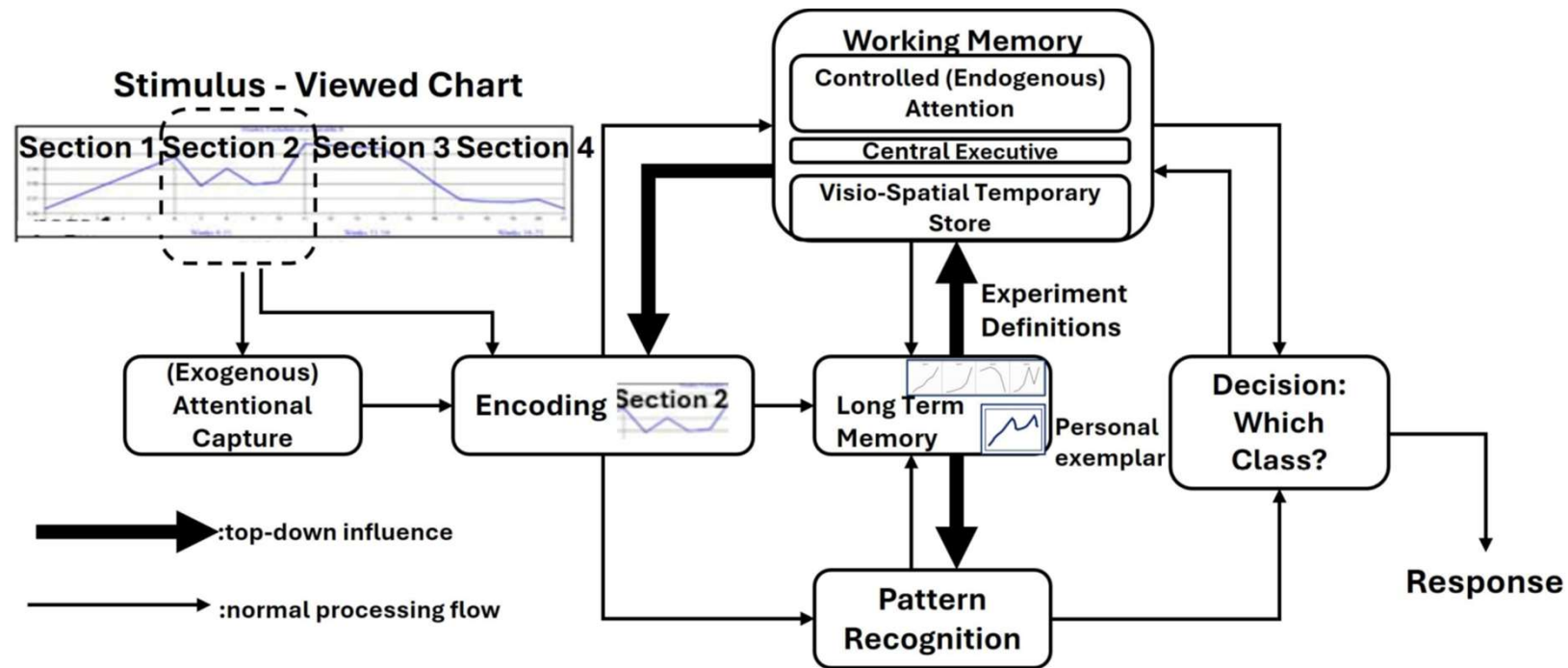


Stevens' Law is about **perceived magnitude $\neq$ actual magnitude**. The key point is: The way we **perceive** differences in length, area, volume, and color does **not scale linearly** with the actual data values.

Type	Actual Data	Physical Scale Used	Perceived Value (Approx)
Length	4 $\rightarrow$ 8 $\rightarrow$ 16	Doubles each time	4 $\rightarrow$ 8 $\rightarrow$ 16 (perceived ~accurate)
Area	4 $\rightarrow$ 8 $\rightarrow$ 16	Area doubles	4 $\rightarrow$ 6.5 $\rightarrow$ 10 (underestimated)
Volume	4 $\rightarrow$ 8 $\rightarrow$ 16	Volume doubles	4 $\rightarrow$ 6 $\rightarrow$ 8 (even more underestimated)
Color	4 $\rightarrow$ 8 $\rightarrow$ 16	Brightness doubles	Hard to distinguish (especially pastel tones)

Mental models/Exemplars

Cognitive framework [1] for classifying patterns in an experiment [2].



[1] R. E. Patterson, L. M. Blaha, G. G. Grinstein, K. K. Liggett, D. E. Kaveney, K. C. Sheldon, P. R. Havig, and J. A. Moore. A human cognition framework for information visualization. *Computers & Graphics*, 42:42– 58, 2014.

[2] S. Eisler, J. Meyer - The classification of patterns in visualizations: effects of data complexity and individual differences – *IEEE Transactions on Visualization and Computer Graphics* (submitted)

5/20/2026

Mental Models

A mental model is a functional analogue representation to an external interactive visualization system with the following characteristics:

- The structural and behavioral properties of external systems are preserved in mental models.
- A mental model can preserve schematic, semantic or item specific information about the underlying data.
- Given a problem, a mental model of an interactive visualization can be constructed and simulated in working memory for reasoning.

Mental Models

- Cognitive Offload: Create new model through insights (symbolic model)
- Visualizations help users offload complex mental processes onto the visual display, freeing up working memory and enabling them to form new, internalized mental models based on insights they discover.

Perceptual Processing

Perception can be:

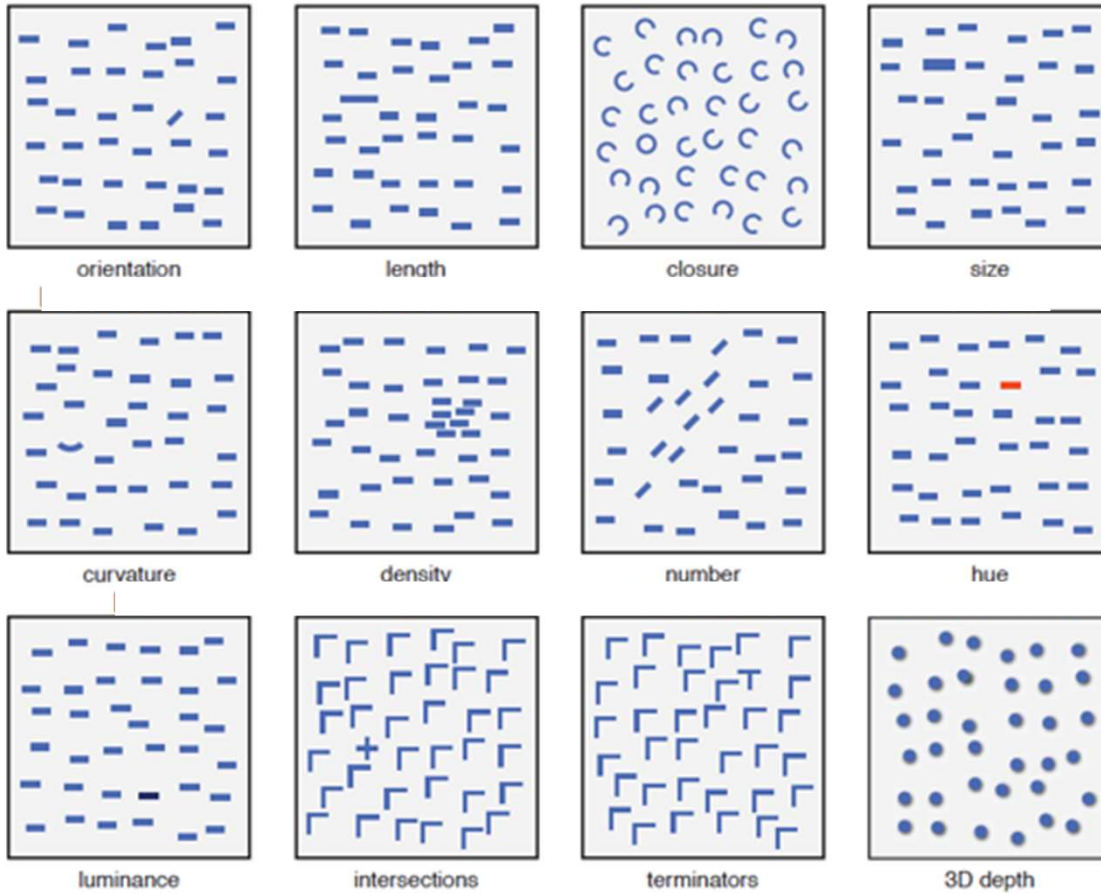
- **intrinsic and uncontrolled (pre-attentive)** fast and parallel, often within 250 ms
- **controlled (attentive)** slower, uses short-term memory, selective and can represent aggregates of what is in the scene

Pre-attentive processing

- Why is this important? In displaying information, it is often useful to be able to show things “at a glance.”
- If you want people to be able to identify instantaneously some mark on a map as being of type A, it should be differentiated from all other marks in a pre-attentive way.

Pre-attentive - Perceptual Processing

- Preattentive processing asks in part, “What visual properties draw our eyes, and therefore our focus of attention to a particular object in a scene?”
- An equally interesting question is, “What do we remember about an object or a scene when we stop attending to it and look at something else?”
- Evidence suggests that a viewer’s current state of mind can play a critical role in determining what is being seen at any given moment, what is not being seen, and what will be seen next.



Healey, C. G., & Enns, J. T. (2012). Attention and Visual Memory in Visualization and Computer Graphics .
 IEEE Transactions on Visualization and Computer Graphics, 18 7), 1170 1188

Feature integration theory (FIT)

- When the human visual system first sees an image, all the features are encoded in parallel into their respective maps. A viewer can access a particular map to check for activity, and perhaps to determine the amount of activity.
- However, the individual feature maps give no information about location, spatial arrangement, or relationships to activity in other maps.
- This framework provides a general hypothesis that explains how pre-attentive processing occurs. If the target has a unique feature, one can simply access the given feature map to see if any activity is occurring. Feature maps are encoded in parallel, so feature detection is almost instantaneous.

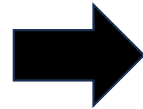
Guided Search Theory - activation map

- An activation map refers to a visual representation of the parts of an image that are most important for a particular task, such as detecting edges or recognizing faces.
- Activation maps are generated by analyzing the responses of neurons in the visual cortex of the brain to specific visual stimuli. These neurons are highly specialized and respond to specific features in the visual scene, such as edges, corners, or colors.

Top-down pattern search and bottom-up inquiry: Guided Search - Wolfe

Effective visual analysis combines bottom-up exploration with top-down guidance

- Bottom-up processing draws attention to interesting features. (Data Driven).
- Top-down processing leverages prior knowledge, goals, and expectations to guide attention, fill in gaps, and interpret what we see. (Goal Driven).



In Data Analysis and Decision Making:

- Help to reveal trends, patterns, relationships, anomalies, and outliers in the data. Support insight generation and exploratory analysis
- Help users make sense of complex data and uncover its underlying structure
- Help building mental models that lead to faster and more confident decisions.

Perceptual Salience

- How should I represent the data?
- How can I highlight important data values when they appear?
- How can I ensure that viewers perceive differences in the data accurately?
- Having a given chart – what is the most salience feature?

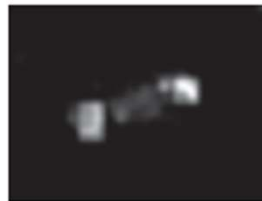
Original Image



Fixation Density Map



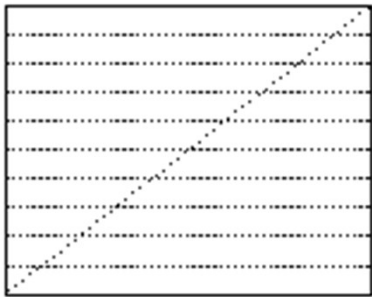
Using local entropy



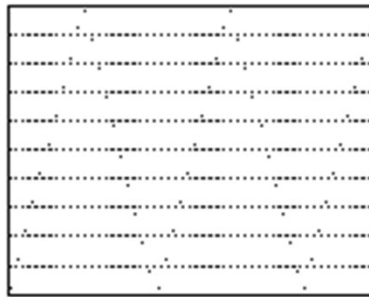
A Computational Model for Saliency Maps by Using Local Entropy
Yuewei Lin - 2010

5/20/2026

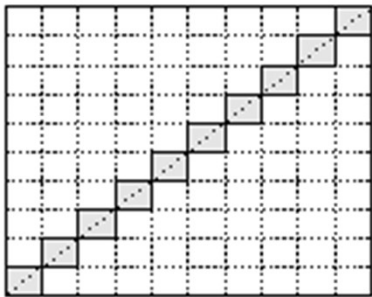
Quantifying Visual Information



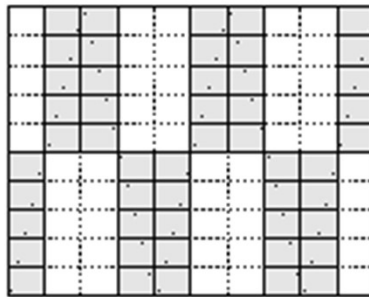
(a)



(b)



(c)



(d)

Use multiple dimensions to calculate entropy. Entropy calculation in multiple bins.

- (a) and (b): the data is divided along the y axis into rows, making a 1D division. For either linear or oscillating data, the measurements are uniformly distributed among 10 bins, producing a relative entropy of 100%.
- (c) and (d): the data is divided along both the x and y axes, making a 2D bin division by dividing both the x and y axes into 10 bins each.

- (c): $H = - \sum p * \log(p) = -10 * \frac{1}{10} * \log(\frac{1}{10})$

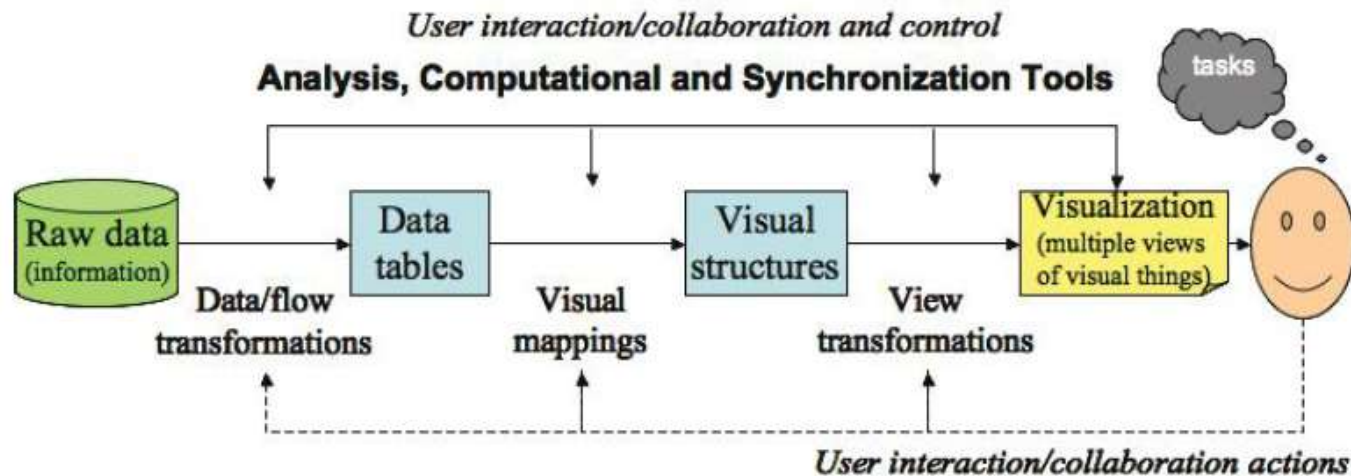
- (d): $H = - \sum p * \log(p) = -50 * \frac{1}{50} * \log(\frac{1}{50})$

- $H_{rel} = \frac{H}{H_{max}}$

$$H_{max} = \log_2(100)$$

Data Sources and Data Processing :Why?

- Remember the Visualization pipeline.
 - Data preprocessing and transformation: process the raw data into something usable by the visualization system
 - Mapping for visualizations: representation mappings: geometry, color.



Types of Data: Ordinal - numeric values

- Binary: assuming only values of 0 and 1
- Discrete: taking on only integer values or from a specific subset (e.g., (2, 4, 6)) – specific days of week, or months, etc.
- Continuous - representing real values: $[0, \infty]$ - time stamps

There is importance to the order of the numbers, it is not just categorical.

Types of Data: Nominal - nonnumeric values

- Categorical (finite list) – Red/Blue/Magenta/Green or yes/no, adult/child
- Ranked - a categorical variable that has an implied ordering (e.g., small, medium, large);
- Arbitrary - a variable with a potentially infinite range of values with no implied ordering (e.g., addresses – names of customers – cities – ids).

Types of Data: Scale

Another method of categorizing variables is by using the mathematical concept of scale:

- Ordering Relation: ranked nominal and ordinal variables
- Distance Metric: distance is computed between different observations (present in all ordinal variables, but is generally not found in nominal variables)
- Existence of absolute zero: weight, kelvin temperature

Types of Data: Scale

- Scale is an important attribute because each graphical attribute that we can control has a scale associated with it.
- The scale of a data variable should be compatible with the scale of the graphical entity or attribute to which it is mapped, but it is also dependent on the task to be performed with the visualization.
- A variable such as *weight* possesses an absolute zero, while *bank balance* does not.

Data Structures: Scalars, Vectors, and Tensors

Data sets have structure: in terms of the means of representation (syntax), and the types of interrelationships within a given record and between records (semantics).

- Scalar: temperature, cost, weight
- Vector: observation has more than 2 values – such as displacement in x and y. Could be also weight and height, color (values of red, green and blue components)
- Tensor: Generic – it is a matrix.

Data Structures: Geometry and Grids

- Mainly in scientific, engineering and geography domains.
- The simplest method of incorporating geometric structure in a data set is to have explicit coordinates for each data record. Example: temperature for longitude and latitude.
- The coordinate system is domain-specific and is partially dependent on how the data is acquired/computed, and on the structure of the space in which the data resides: Cartesian, Spherical ,Hyperbolic coordinates
- Nonuniform (irregular) geometry:

Data Structures: Geometry and Grids

Nonuniform (irregular) geometry:

The Earth's Surface (General Riemannian Geometry)

- Consider the Earth's surface as a 2D surface embedded in 3D space.
- While it's roughly spherical, the actual curvature **varies** depending on local geography—mountains, valleys, and other terrain features.
- This is an example of a **nonuniform (irregular) geometry**, because:
 - The **metric** (which tells you how to measure distance) changes from place to place.
 - The **curvature** is not constant.

Data Structures: Hyperbolic Coordinates

Domain: UX, InfoVis, hierarchical data (like file systems, org charts, etc.)

Use: When you want to display a huge hierarchy in a compact space.

Why hyperbolic? In hyperbolic space, **more room is available near the edges** than in Euclidean space. So you can fit more nodes with less overlap.

Visualization style:

- Nodes are placed using **hyperbolic geometry**.
- As you zoom/pan, the focus shifts and "nearby" nodes appear larger, while distant ones shrink and move toward the edge of the circle.

Famous example:

- **Hyperbolic browser** by Lamping et al. (1995): used to navigate large hierarchies by mapping them into hyperbolic space.
- *Imagine a circular view where the center shows a zoomed-in portion of the hierarchy, and the rest shrinks out toward the edge.*

Hyperbolic Geometry Coordinates (in curved space)

The hyperbolic browser by Lamping, Rao, and Pirolli (1995) uses a brilliant trick:

1. Lays out a tree in hyperbolic space (usually the Poincaré disk model) and lets you interactively explore by smoothly transforming the layout as you change the focus.
2. Points are represented in the Poincaré disk model (2D unit disk)
3. Transformations - especially focus changes are done via Möbius transformations.
4. Lets see example:
https://www.youtube.com/watch?v=J0yFd2Pq_rg&t=6s (Visualizing Large Trees Using the Hyperbolic Browser)

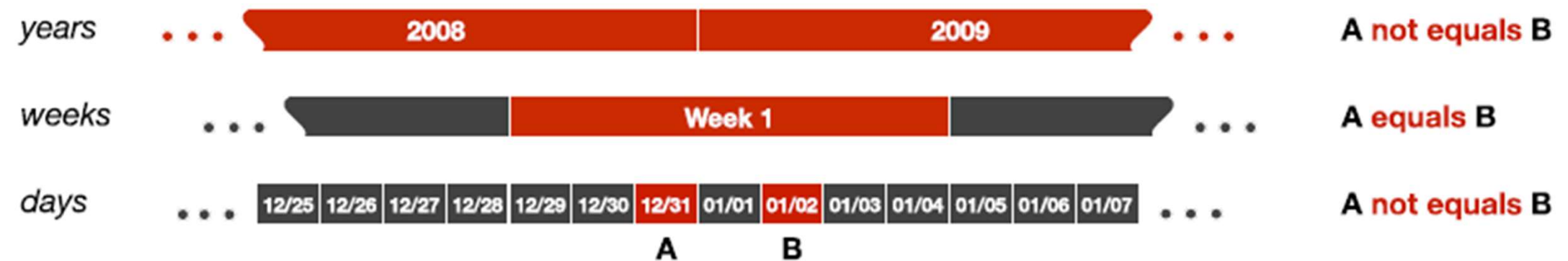
Data Structures: time series

- Granularity
- Hierarchies – for aggregation and comparison, drill down
- Span – for filtering, slice and dice
- Seasonality – for comparison

Data Structures: time series

- Very common: Financial/business. No geometric structure, n possibly independent components, nominal and ordinal, with a temporal attribute
- Data sets with temporal attributes can be uniformly spaced, such as in the sampling of a continuous phenomenon, or nonuniformly spaced, as in a business transaction database.

Data Structures: time series



Granularities influence equality relationships. The times of A and B are not equal on the granularity of days, but are equal on the granularity of weeks, and then again are not equal on the coarser granularity of years.

Aigner: Visualization of Time-Oriented Data – page 56

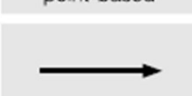
Data Structures: time series

Two dimensions:

- Time
- Something else (measure, event, etc.)
- Time is part of the data, but it is not the data. It is how the data is ordered.

Data Structures: time series

Time

scale	 ordinal	 discrete	 continuous
scope	 point-based	 interval-based	
arrangement	 linear	 cyclic	
viewpoint	 ordered	 branching	 multiple perspectives
Abstractions			
granularity & calendars	 none	 single	 multiple
time primitives	 instant	 interval	 span
determinacy	 determinate	 indeterminate	

Cyclic time: Set of recurring time values such as the seasons of the year.

Viewpoint: ordered vs. branching vs. multiple perspectives

Time primitives: instant, interval, and span.

Determinacy: determinate vs. indeterminate: Uncertainty

Design aspects of time-oriented data

5/20/2026

Source: Visualization of Time-Oriented Data, Aigner 2011

Data Structures: time series

Data

scale	3,14 3,27 4,88 quantitative	coconut banana apple qualitative
frame of reference	▼ abstract	🌐 spatial
kind of data	⌈ ⌋ events	— — states
number of variables	📈 univariate	📉 multivariate

Design aspects of time-oriented data

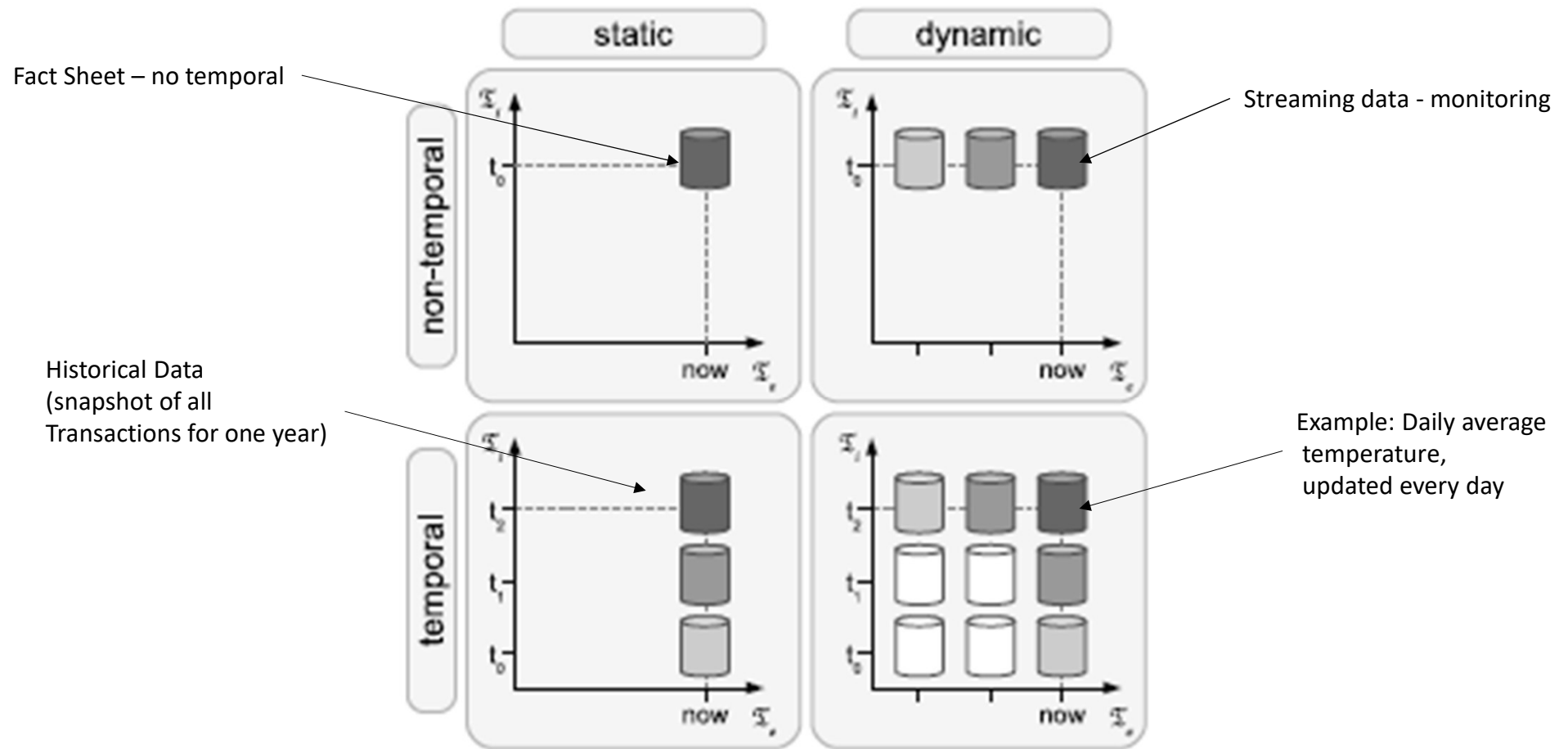
5/20/2026

Source: Visualization of Time-Oriented Data, Aigner 2011

Data Structures: time series

- Examples of time primitives – span in Gant for example...
- Multiple perspectives: Vincent was born on August 8, 2006 (valid time) and this fact was stored in the register of residents two days later on August 10, 2006 (transaction time).
- Branching time. Alternative scenarios for moving into a new living space.
- Internal time describes when the information contained in the data is valid. Conversely, external time is considered to be extrinsic to the data model. The external time is necessary to describe how a dataset evolves in (external) time.
- Health data or climate data that contain measures depending on time (e.g., daily number of cases of influenza or daily average temperature), and that are updated every 24 hours with new data records of the passed day
- Slowly changing dimensions....??

Data Structures: time series



Data Structures - Topology

Topology – how the data records are connected:

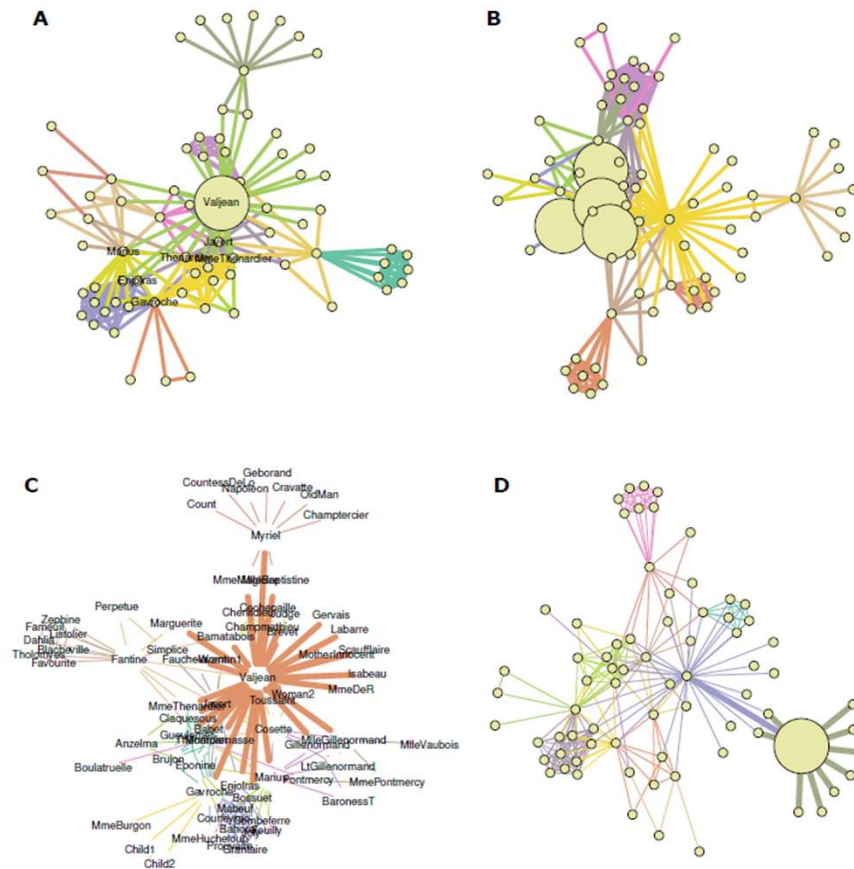
- Financial (business): No geometric structure, n possibly independent components, nominal and ordinal, with a temporal attribute
- Social Network: Nodes consisting of multiple fields of all types, with various connectivity attributes that could be spatial, temporal, or dependent
- MRI (magnetic resonance imagery): Density (scalar), with three spatial attributes, 3D grid connectivity

Data Structures - Topology

Topology – how the data records are connected:

- CFD (computational fluid dynamics): Three dimensions for displacement, with one temporal and three spatial attributes, 3D grid connectivity (uniform or nonuniform)
- CAD (computer-aided design): Three spatial attributes with edge and polygon connections, and surface properties
- Remote sensing – IOT: Multiple channels, with two or three spatial attributes, one temporal attribute, and grid connectivity
- Census: Multiple fields of all types, spatial attributes (e.g., addresses), temporal attribute, and connectivity implied by similarities in fields

Data Structures - network visualizations



The generation, visualization, and analysis of link communities in arbitrary networks with the R package linkcomm, Alex T. Kalinka 10`4

Data Processing

- Similar to BI (Business Intelligence) ETL (Extract/Transform/Load) and ELT
- Metadata – is information about the data

Data Processing: Missing Values and Data Cleansing

- Discard the bad record – risky because of information loss
- Assign a sentinel value and visualize – example: in a variable that has a range of 0 to 100, one might use a value such as -5 to designate an erroneous or missing entry. Then visualize so that the records with problematic data entries will be clearly visible.
- Assign the average value – risk: average may not be a good guess
- Assign value based on nearest neighbor

In all cases where a value is substituted for the missing or erroneous value, it is critically important that the fact that this value is “suspect” be preserved, and that **any visualization of this data must convey this information.**

Data Processing

- Statistical analysis – is important to understand how to show the data
 - Mean, maximum, minimum, standard deviation
 - Outliers
 - Errors
 - Correlation analysis

Data Processing: Normalization

Normalization – transforming data so that the results satisfy a particular statistical property. It must be a 1 to 1 mapping.

- Most common: $y_n = \frac{(y_{original} - y_{min})}{(y_{max} - y_{min})}$
- Square root mapping: $y_n = \frac{(\sqrt{y_{original}} - \sqrt{y_{min}})}{(\sqrt{y_{max}} - \sqrt{y_{min}})}$
- Logarithmic mapping: $y_n = \frac{(\log(y_{original}) - \log(y_{min}))}{(\log(y_{max}) - \log(y_{min}))}$

Data Processing: Normalization

- To ease interpretation, we sometimes choose the scaling and offset factors to coincide with intuitive maximum and minimum values, rather than the ones the data possess. For example, if the data corresponds to percentages that fall between 40 and 90, we might use the full range of 0 to 100 instead.
- Note that such a mapping changes the data values and distribution, and it is **essential that it is communicated to the user in the visual mapping.**
- Normalization may also involve bounding values, so that, for example, values exceeding a particular threshold are capped at that threshold.

Data Processing

- Segmentation can help – using clusters
- Dimensions reduction – machine learning techniques
 - principal component analysis (PCA)
 - Multidimensional scaling (MDS)
- Detrending
- Time Series Decomposition: term trend, seasonal fluctuations, and remainder

Data Processing - Dimensions reduction

- Principal component analysis (PCA): The PCs are chosen such that they are uncorrelated, and they are ordered such that the first component captures the largest possible amount of variation in the data and subsequent components capture increasingly less. Usually, key features in the data can be seen from only the first two or three PCs.
- Multidimensional scaling (MDS): tries to find a lower dimensional representation for a data set that best preserves the inter-point distances from the original data set.

It is important to note that most of these techniques do not produce unique results. Starting configurations, parameters of the calculations, and variations in the computations can lead to quite different results.

Data Processing - PCA

Reducing a four-dimensional data set (Iris) to two dimensions using PCA and plotting the resulting points.

A form of glyph is used, called a star glyph, to convey the original data points.

The PCA algorithm does a good job of separating data into clusters.

The Iris data set in star glyphs, with the position of each point based on the first two principal components. The star glyph represents four variables as the lengths of the each of the lines emanating from the center of a four-pointed star. Reasonable clustering can be seen.

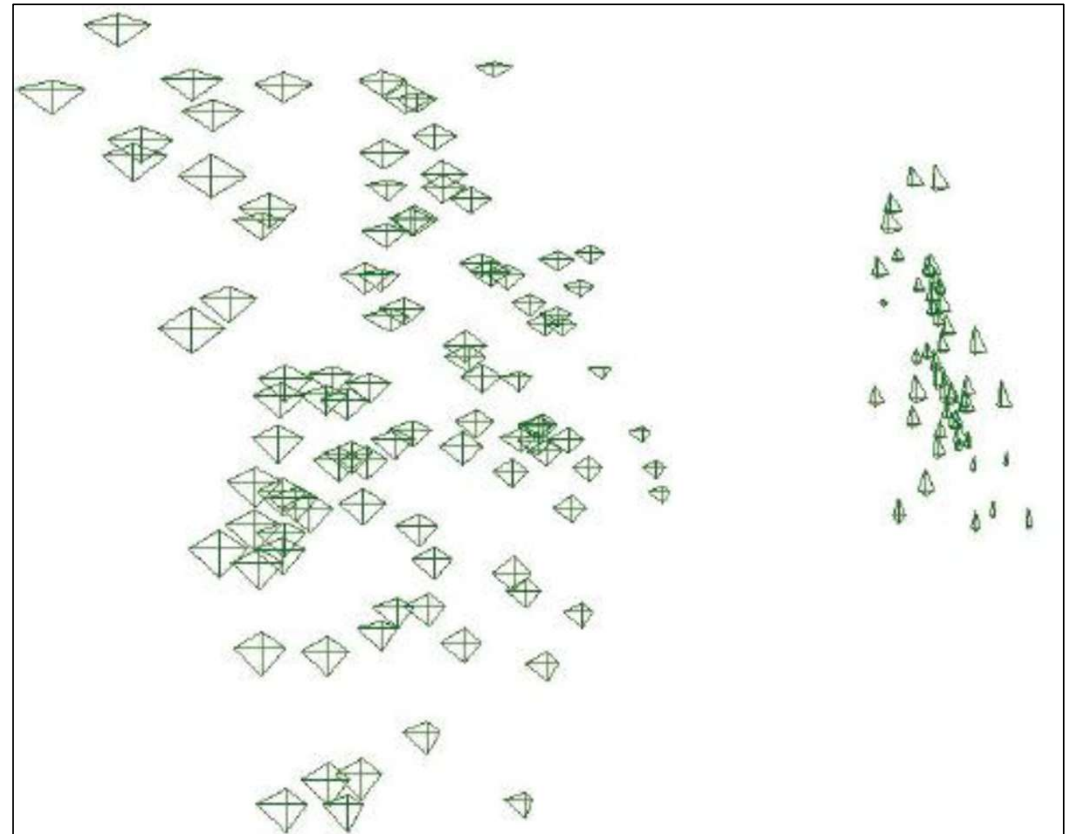


Figure 2.4. - Ware

Data Processing – PCA - explanation

The data: The Iris dataset is a classic dataset with 4 numerical features (variables) per data point:

- Sepal length
- Sepal width
- Petal length
- Petal width

So, each data point is originally in 4D space.

Data Processing – Dimensionality Reduction with PCA:

We can't directly plot 4D data, so Principal Component Analysis (PCA) is used to reduce the dimensionality to 2D while preserving as much variance (information) as possible.

- The **X and Y position** of each point in your plot comes from the **first two principal components**.
- This is just like mapping high-dimensional data into a 2D coordinate system for visualization.

So, imagine each flower is now represented as a dot in this 2D space based on its features.

Data Processing – What Are Star Glyphs?:

Each data point (each flower in the Iris dataset) is not just a dot — it's shown using a **star glyph**.

- A **star glyph** is a kind of small "chart" where:
 - Each "arm" or "spoke" represents one variable (here, one of the 4 features).
 - The length of the arm represents the value of that variable for that data point.
 - In our case, we have 4 spokes (since you have 4 features).
- Think of it like a mini radar chart or spider plot drawn at each point.

Data Processing - PCA

How to Read the Image:

1.Position of each glyph: shows the result of PCA → a summary of the overall shape of the data.

2.Shape of the glyph (star): shows the detailed values of the original four features.

3.Cluster on the right: these data points are more similar to each other in terms of the principal components.

4.Differences in star shapes: reflect differences in sepal/petal lengths and widths.

For example:

- A long spoke to the right might mean a high petal length.
- A short spoke up might mean a small sepal width.

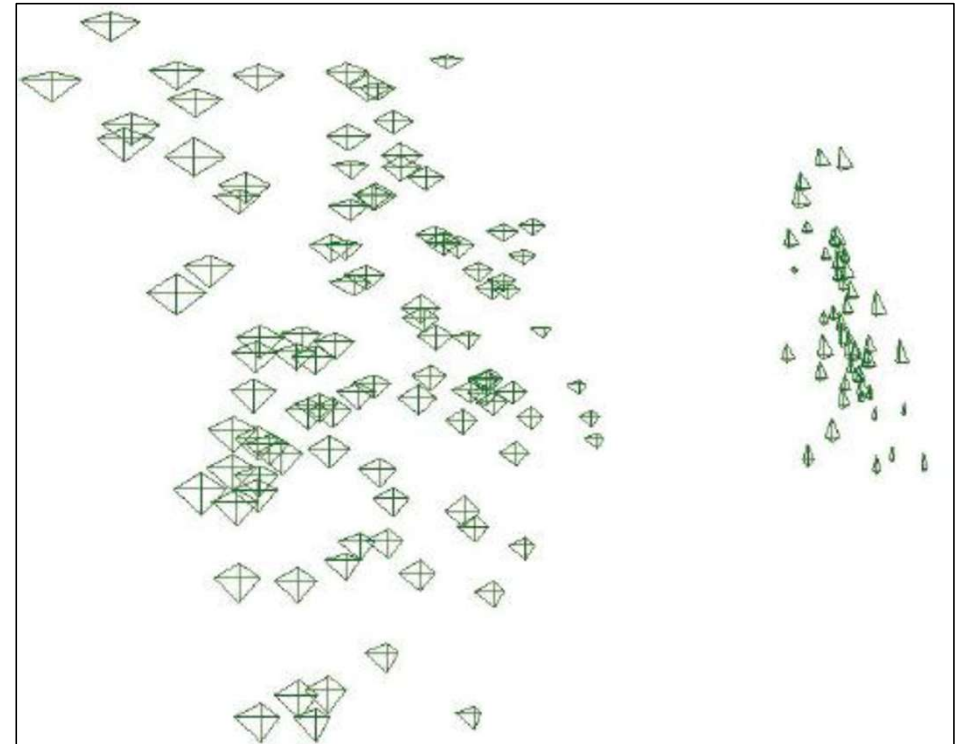


Figure 2.4. - Ware

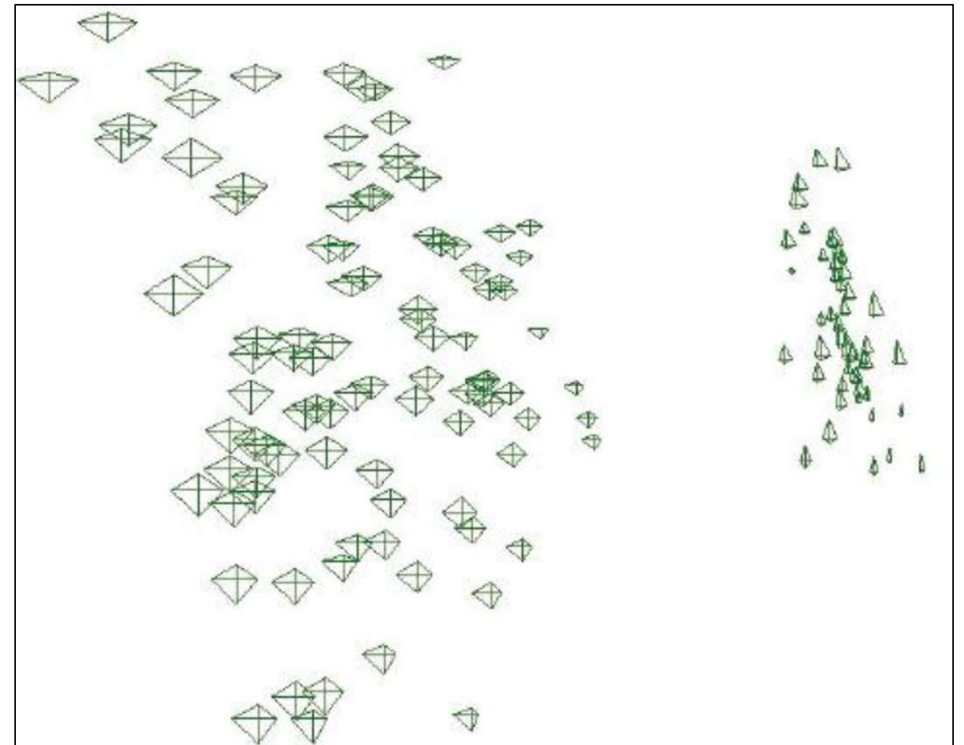
Data Processing - PCA

1. The Left Image (Larger spread of glyphs):

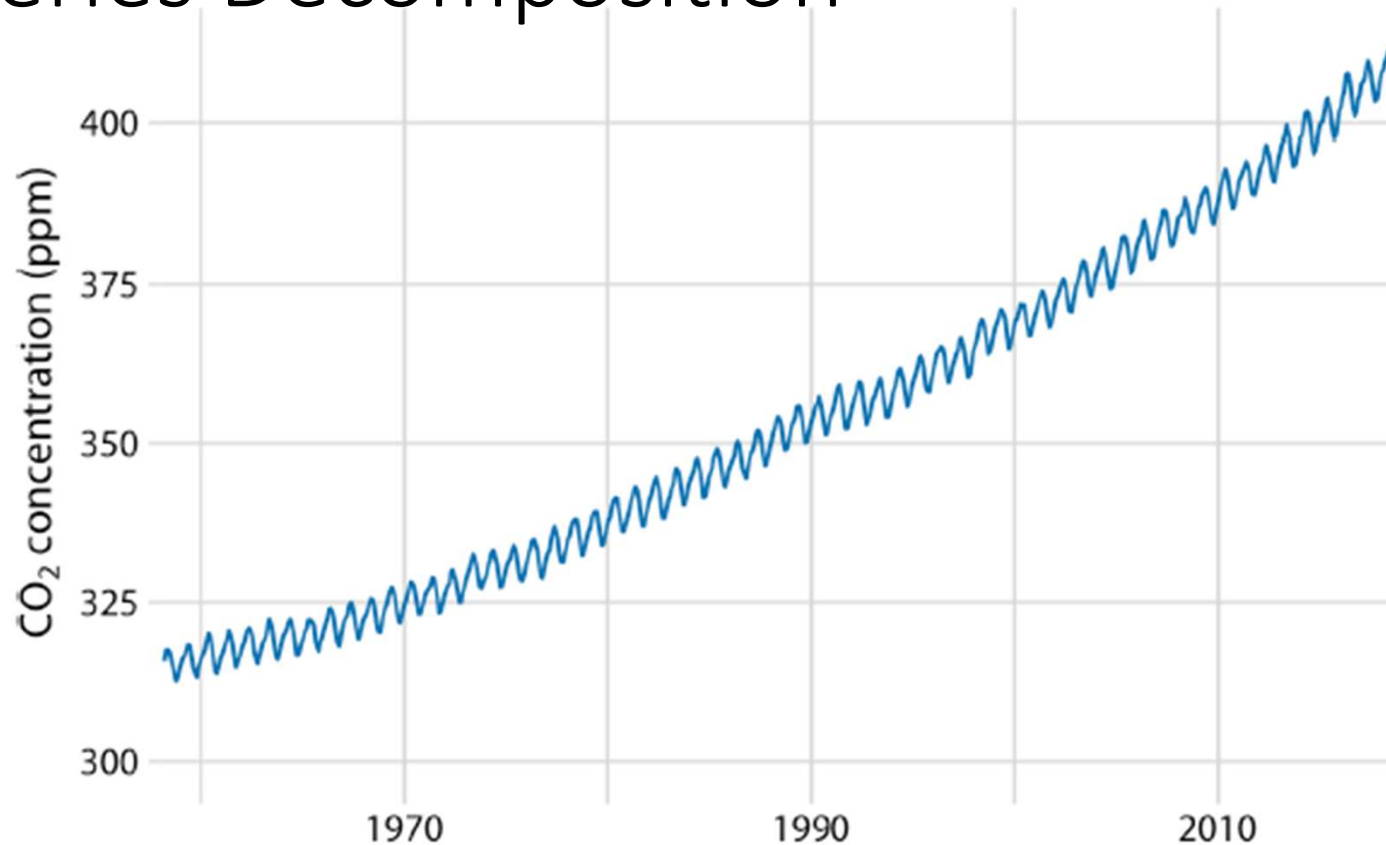
- **This is the entire Iris dataset**, plotted using PCA (first two components). Projected into 2D, with all the details kept via the glyphs.
- Each **point is a star glyph**, and its **position** is based on the values of **PC1 and PC2**.
- The **glyph shape** at each point shows the original 4 features (sepal/petal length/width).

2. The Right Image (Smaller group of glyphs):

- This is likely a **subset** of the dataset — probably **one of the Iris species** (e.g., *Iris setosa*).
- These points are **clustered together** because:
 - In PCA space, this species is well-separated from the others.
 - The glyphs are similar in shape, showing consistent feature values.
- This clearly demonstrates **how PCA separates groups**, and the glyphs show **why** (e.g., smaller petal width and length for setosa).



Time Series Decomposition

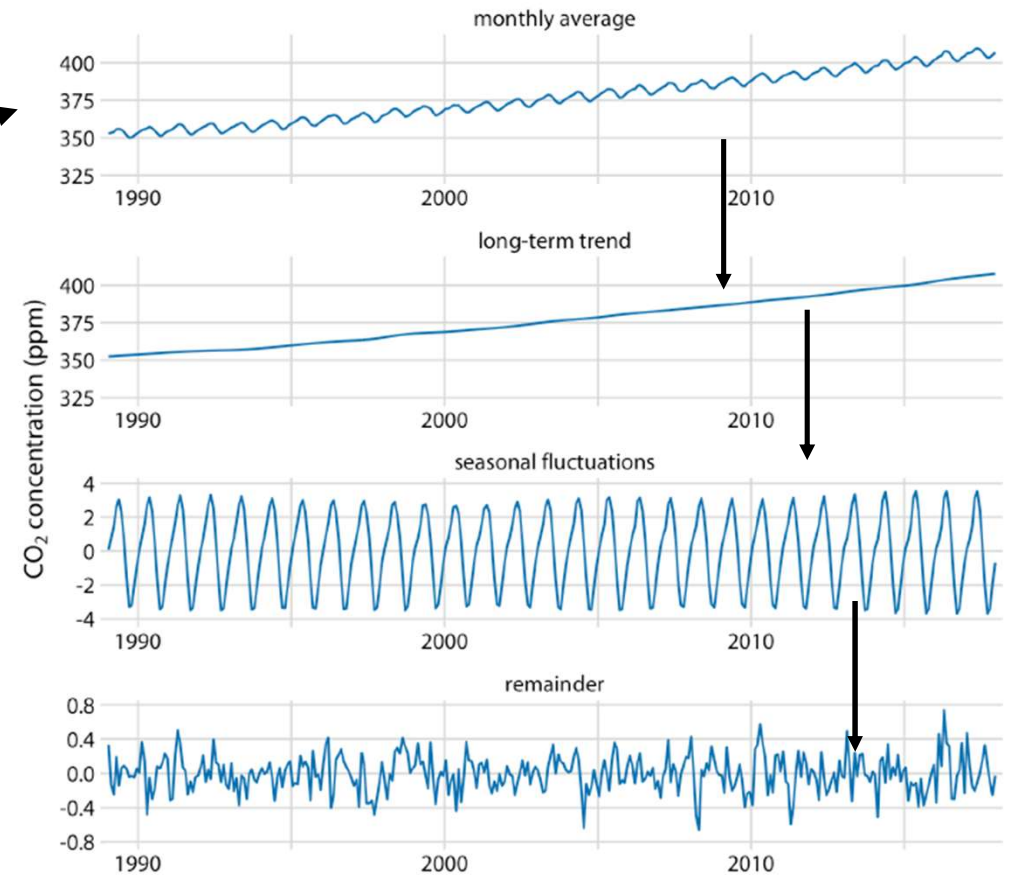
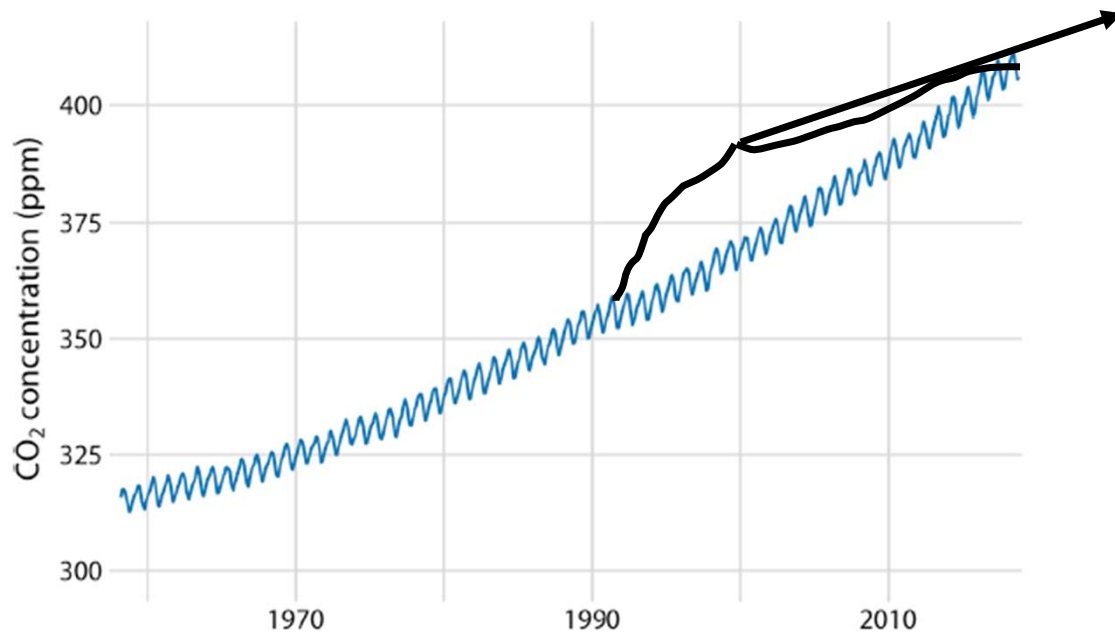


The Keeling curve. The Keeling curve shows the change of CO₂ abundance in the atmosphere over time

Fundamentals of Data Visualization A Primer on Making Informative and Compelling Figures - Claus O. Wilke – pages 158-9

5/20/2026

Time Series Decomposition

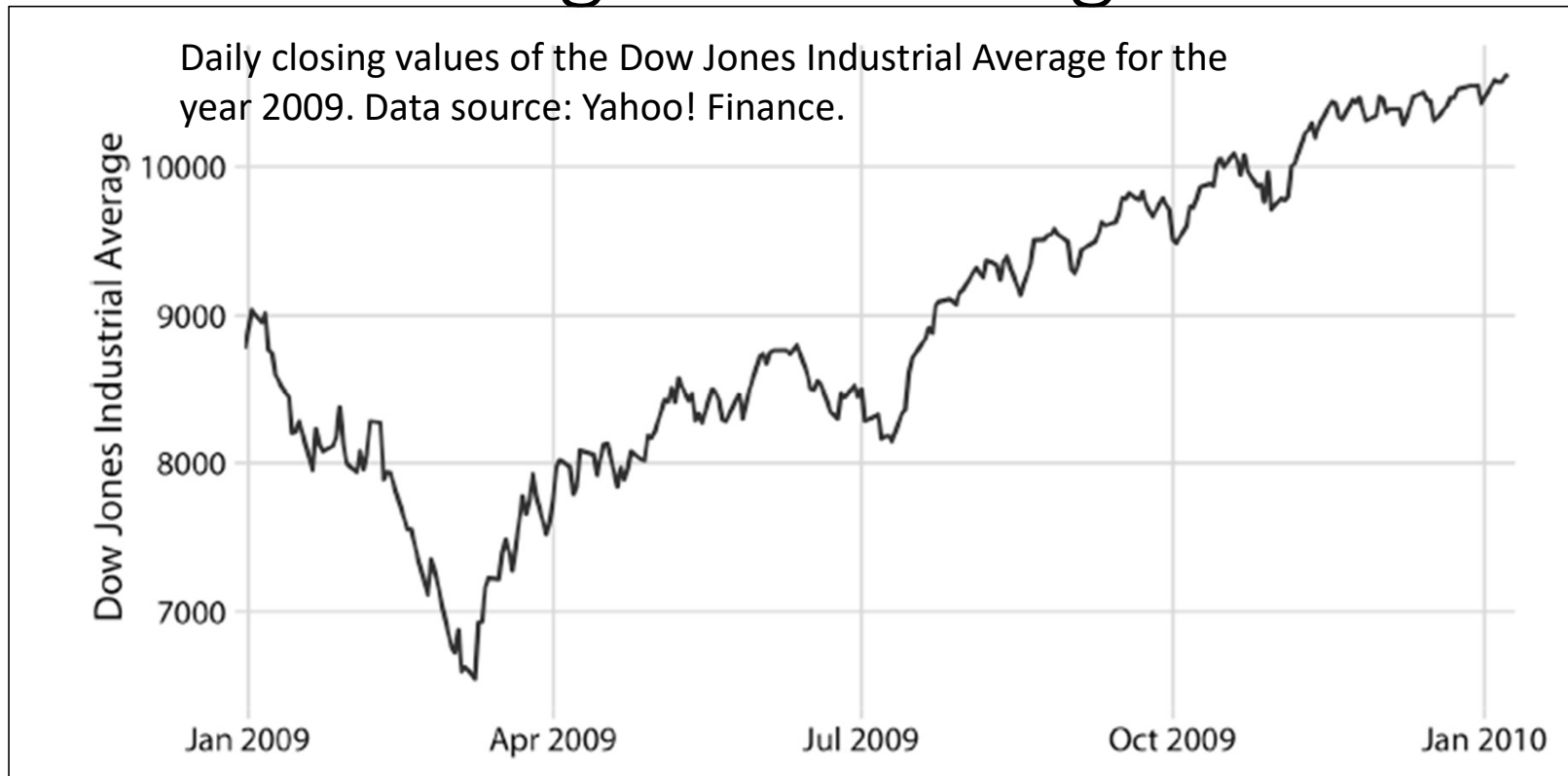


Data source: Dr. Pieter Tans, NOAA/ESRL, and Dr. Ralph Keeling,
Scripps Institution of Oceanography,

Data Processing

- Mapping Nominal Dimensions to Numbers. The key is to find a mapping of the data to a graphical entity or attribute that doesn't introduce artificial relationships that don't exist in the data
- Aggregation and Summarization. The key to visually depicting aggregated data is to provide sufficient information for the user to decide whether he or she wishes to perform a drill-down on the data.

Data Processing - Smoothing and Filtering



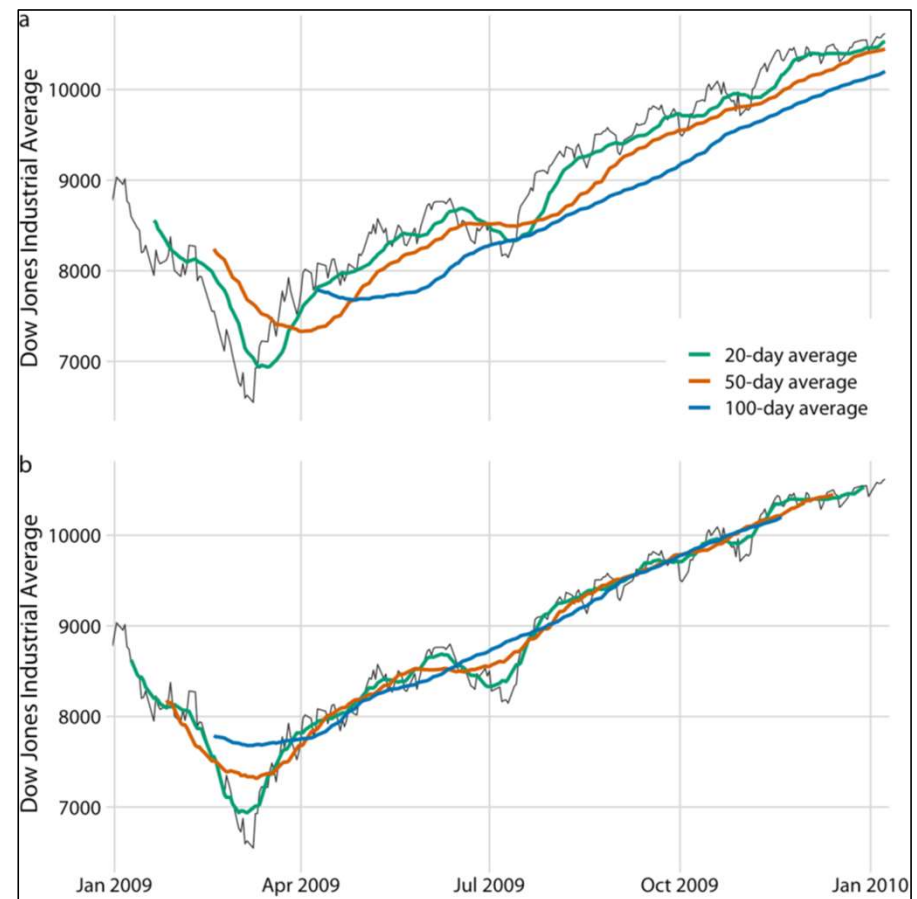
Fundamentals of Data Visualization A Primer on Making Informative and Compelling Figures - Claus O. Wilke – pages 146-7

Data Processing - Smoothing and Filtering

- Smoothing produces a function that captures key patterns in the data while removing irrelevant minor detail or noise.
- Financial analysts usually smooth stock market data by calculating moving averages. They often plot each average at the end of its respective time window. This choice results in curves that lag the original data (Figure 14-2a), with more severe lags corresponding to larger averaging time windows.

Fundamentals of Data Visualization A Primer on Making Informative and Compelling Figures - Claus O. Wilke – pages 146-7

5/20/2026

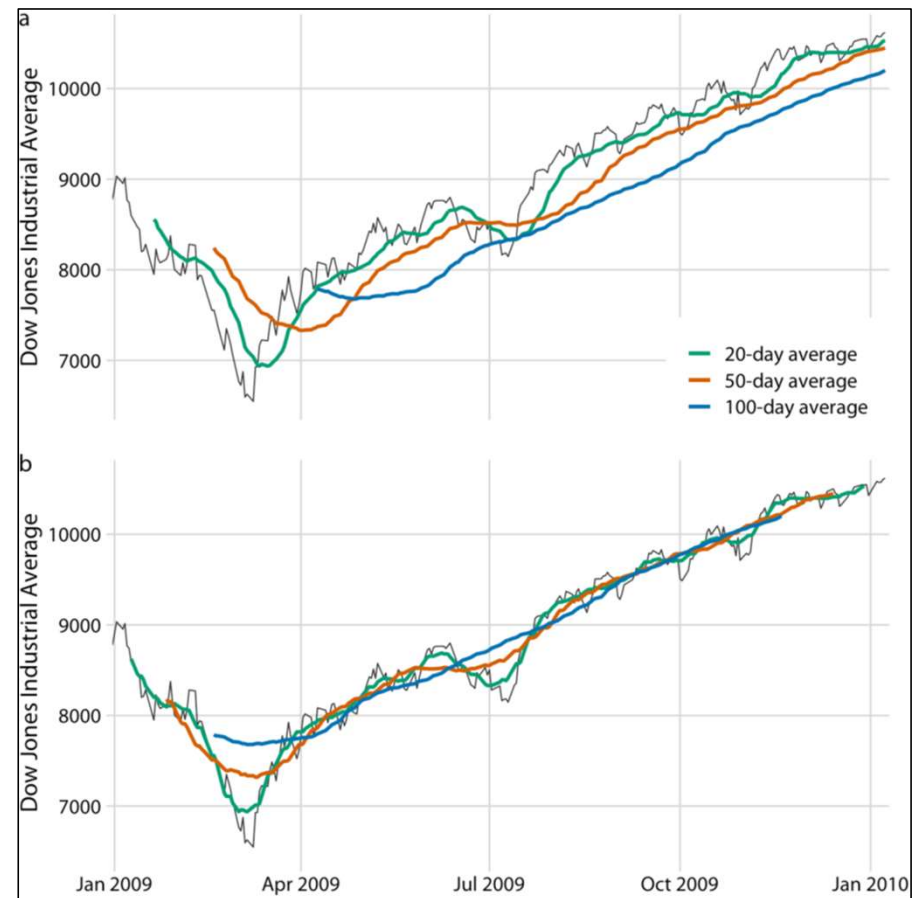


Data Processing - Smoothing and Filtering

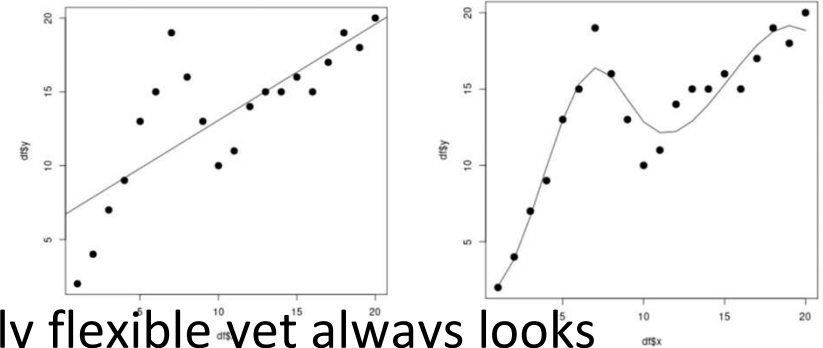
- Statisticians, on the other hand, plot the average at the center of the time window, which results in a curve that overlays perfectly on the original data.
- The 20-day moving average removes small, short-term spikes but otherwise follows the daily data closely. The 100-day moving average, on the other hand, removes even fairly substantial drops or spikes that play out over a time span of multiple weeks.

Fundamentals of Data Visualization A Primer on Making Informative and Compelling Figures - Claus O. Wilke – pages 146-7

5/20/2026



Data Processing - Smoothing



- Spline : piecewise polynomial function that is highly flexible yet always looks smooth
- Local regression or local polynomial regression, also known as moving regression, is a generalization of moving average and polynomial regression. Its most common methods, initially developed for scatterplot smoothing, are LOESS (locally estimated scatterplot smoothing) and LOWESS (locally weighted scatterplot smoothing). From Wikipedia.
- LOESS is not limited to time series
- Many different types of splines, including cubic splines, B-splines, thin-plate splines, Gaussian process splines, and many others, and which one to pick may not be obvious

Example of R code: <https://naturegeorge.github.io/eigenblog/posts/lowess/>

Data Processing

- Showing Trends with a Defined Functional Form – models
 - It is usually better to fit a straight line to transformed data than to fit a nonlinear curve to untransformed data.
 - Avoid exponential fits and instead use linear fits on log-transformed data
- This reduces the risk of misleading extrapolation
- Makes trends more perceptually accessible and visually interpretable



Example 1: Visualizing Exponential Growth (e.g., COVID-19 Cases)

Bad Practice:

- Plot cumulative case counts over time
- Fit an **exponential curve** to the raw data: $y = a * e^{(bx)}$ (harder to interpret visually, prone to overfitting)

Better Practice:

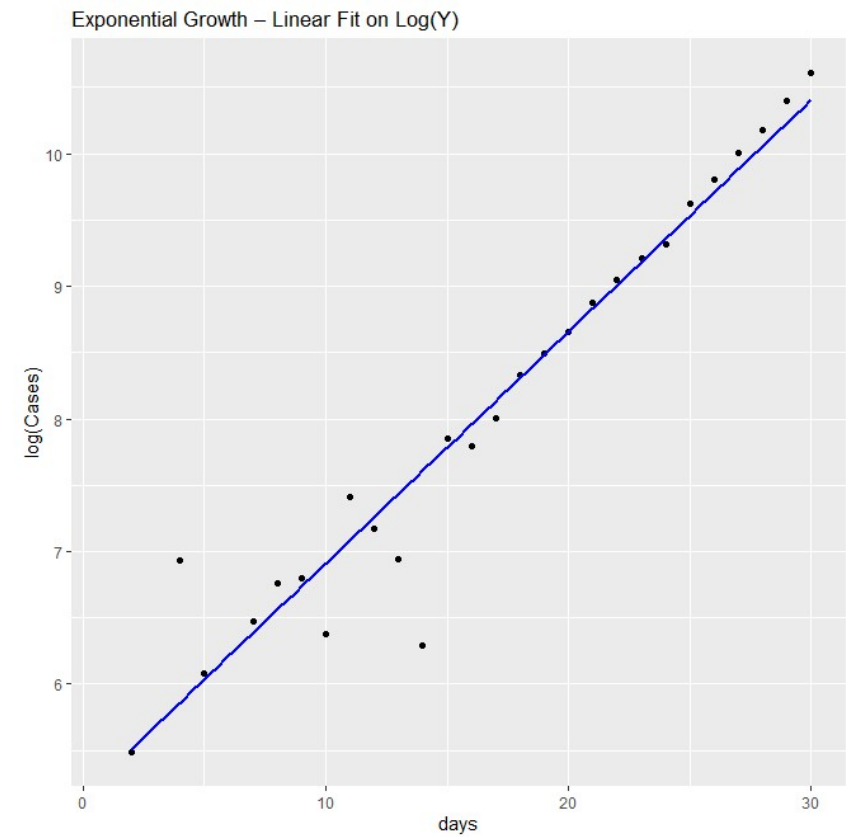
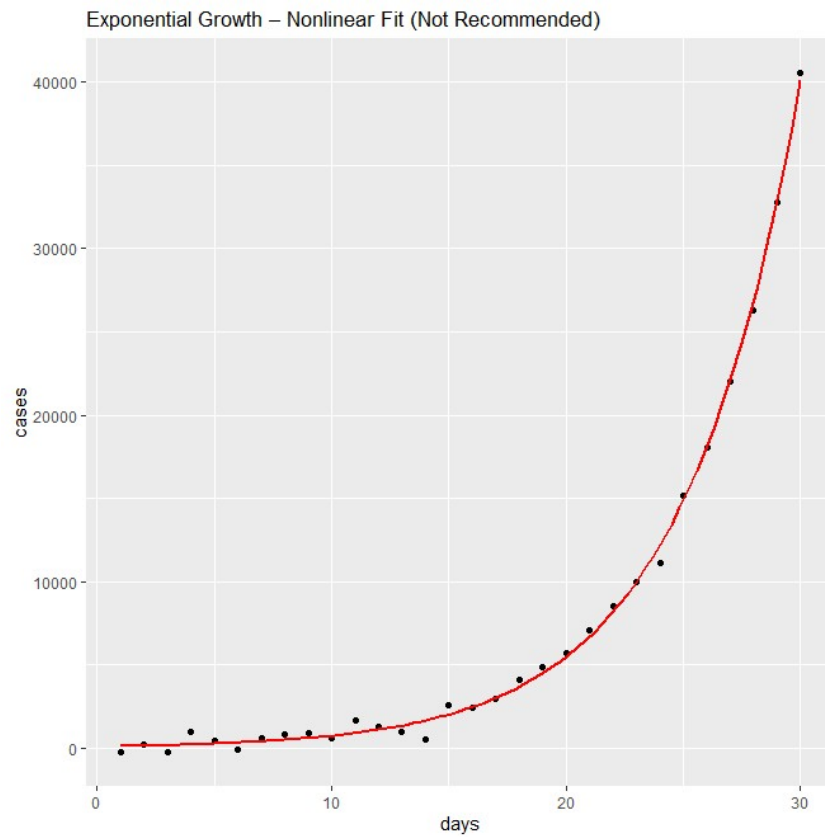
- Apply a **log transformation** to the y-axis:
 - Plot $\log(y)$ vs. time (x)
 - Fit a **straight line**: $\log(y) = \log(a) + bx$

Why it's better:

- Log-transformed data often linearizes exponential trends
- Easier to spot deviations (e.g., slowing of growth)
- Linearity is easier to perceive and interpret visually



Example 1: Visualizing Exponential Growth (e.g., COVID-19 Cases)





Example 1: Visualizing Exponential Growth (e.g., COVID-19 Cases)

Simulated exponential data

```
set.seed(1)
```

```
days <- 1:30
```

```
cases <- 100 * exp(0.2 * days) + rnorm(30, 0, 500)
```

```
df_exp <- data.frame(days, cases)
```

Plot raw with nonlinear curve (bad)

```
p1 <- ggplot(df_exp, aes(x = days, y = cases)) +
```

```
  geom_point() +
```

```
  geom_smooth(method = "nls", formula =  $y \sim a * \exp(b * x)$ ,
```

```
    method.args = list(start = list(a = 100, b = 0.2)),
```

```
    se = FALSE, color = "red") +
```

```
  labs(title = "Exponential Growth – Nonlinear Fit (Not Recommended)")
```

Plot log-transformed y with linear fit (better)

```
p2 <- ggplot(df_exp, aes(x = days, y = log(cases))) +
```

```
  geom_point() +
```

```
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
```

```
  labs(title = "Exponential Growth – Linear Fit on Log(Y)",
```

```
    y = "log(Cases)")
```

Example 2: Moore's Law (Semiconductor Transistor Counts)

Original:

- Transistor counts double approximately every two years — exponential growth

Better Visualization:

- Use a log scale on the y-axis
- Then fit a linear regression line to the transformed data

Why it's better:

- This clearly shows the consistent doubling pattern over time, and any departure from the trend becomes visually obvious.

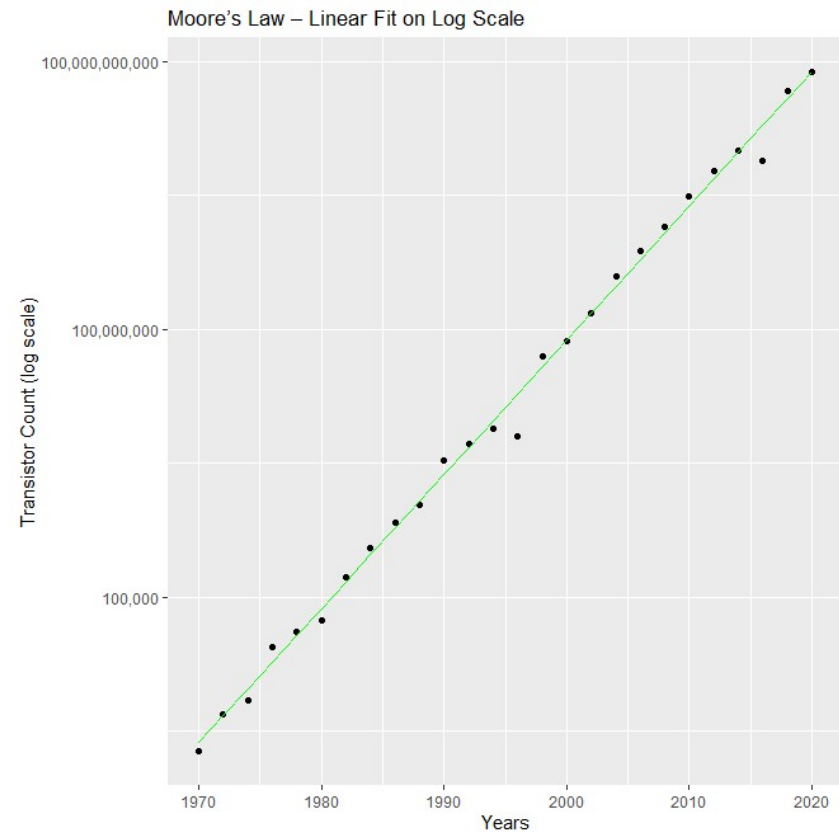


Example 2: Moore's Law (Semiconductor Transistor Counts)

Black dots: the real data (transistor counts)

Green line: the model's prediction, properly aligned with the log-scaled y-axis

```
library(ggplot2)
library(dplyr)
library(scales)
set.seed(1)
# Simulated Moore's Law data (transistors doubling every 2 years)
years <- seq(1970, 2020, by = 2)
transistors <- 2300 * 2^((years - 1970)/2) * (1 + rnorm(length(years), 0, 0.3))
df_moore <- data.frame(years, transistors)
# Fit linear model on log10 scale
model <- lm(log10(transistors) ~ years, data = df_moore)
# Back-transform the predictions to the original scale
df_moore$predicted <- 10^predict(model)
# Plot
ggplot(df_moore, aes(x = years)) +
  geom_point(aes(y = transistors)) +
  geom_line(aes(y = predicted), color = "green") +
  scale_y_log10(labels = comma) +
  labs(title = "Moore's Law – Linear Fit on Log Scale",
       y = "Transistor Count (log scale)",
       x = "Years")
```



back-transform the predicted log values and then plot 68

Assignment – previous year

1. The problem (has to be related to Data Science – to something you can model or investigate).
 - a) Explain what is the problem that the interactive app will help solving
 - b) Explain how will the app help solving the problem
2. The data:
 - a) What is the data source
 - b) Data structure and type of data
 - c) Statistics, outliers, trends (use visualizations)
3. Data transformations:
 - a) Explain what data transformation were used in order to visualize the data in the app or process
 - b) Why were those transformations used?
4. Interactions and visualizations:
 - a) General flow of the interaction
 - b) Explanation on the interactions and the visualizations, how they work and what the user does
 - c) Explain why these interactions and these visualizations
 - d) Include copy of the code
5. Working application – demo – 10 /15 minutes

Assignment - new

•Problem Definition

- Define a Data Science / analytics problem
- Explain target users and decision-support goals
- Explain how the interactive app helps solve the problem

•Data Understanding

- Data source and structure
- Exploratory analysis (trends, outliers, patterns)
- Visual summaries and insights

•Data Processing

- Cleaning, transformations, feature engineering
- Explain why transformations were needed

•Visualization & Interaction Design

- Explain visualization choices
- Describe interactions and user flow
- Justify design decisions

•AI Usage Documentation (NEW)

- List AI tools used (ChatGPT, Claude, Copilot, etc.)
- Document important prompts used
- Specify which AI capabilities/skills were used:
 - Code generation
 - Debugging
 - UI design
 - Data analysis
 - Visualization suggestions
 - Refactoring
 - Documentation
 - Idea generation
- Explain how AI outputs were validated/corrected

•Evaluation & Reflection

- Strengths and limitations of the system
- What AI helped with vs. human decisions
- Lessons learned and future improvements

•Deliverables

- Report
- Source code
- AI prompt appendix
- Working demo (10–15 min presentation)

Public Data Sets

- Section 2.4 (page 73 of the book - Ware)
- Kaggle: <https://www.kaggle.com/datasets>
- NASDAQ: <https://www.nasdaq.com/market-activity/index/comp/historical>
- Etc.

R for next class

- Install R Windows - <https://cran.r-project.org/bin/windows/base/>
- Install R Mac - <https://cran.r-project.org/bin/macosx/>
- Install R Studio - <https://posit.co/download/rstudio-desktop/>